

INTRODUCTION TO PROGRAMMING

UNIT-3

ARRAYS

Using Arrays in C

C supports a derived data type known as array that can be used to handle large amounts of data (Multiple values) at a time.

Definition:

An array is a group (or collection) of same data types.

Or

An array is a collection of data that holds fixed number of values of same type.

Or

Array is a collection or group of elements (data). All the elements of array are homogeneous (similar). It has contiguous memory location.

Or

An array is a data structured that can store a fixed size sequential collection of elements of same data type.

What's the need of an array?

Suppose you have to store marks of 50 students, one way to do this is allotting 50 variables. So it will be typical and hard to manage. For example we cannot access the value of these variables with only 1 or 2 lines of code.

Another way to do this is array. By using array, we can access the elements easily. Only few lines of code is required to access the elements of array.

Where arrays are used

- to store list of Employee or Student names,
- to store marks of a students,
- Or to store list of numbers or characters etc.

Advantage of C Array

- 1) Code Optimization: Less code to access the data.
- 2) Easy to traverse data: By using the for loop, we can retrieve the elements of an array easily.
- 3) Easy to sort data: To sort the elements of array, we need a few lines of code only.
- 4) Random Access: We can access any element randomly using the array.

Disadvantage of Array

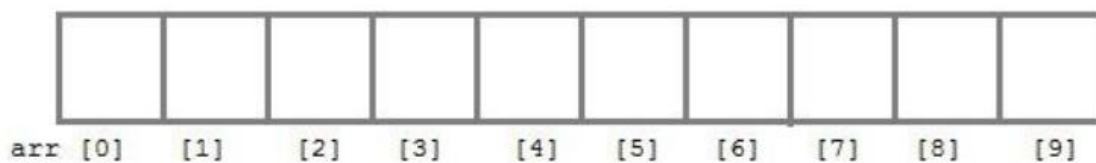
Fixed Size: Whatever size, we define at the time of declaration of array, we can't exceed the limit. So, it doesn't grow the size dynamically like Linked List

Declaration of an Array

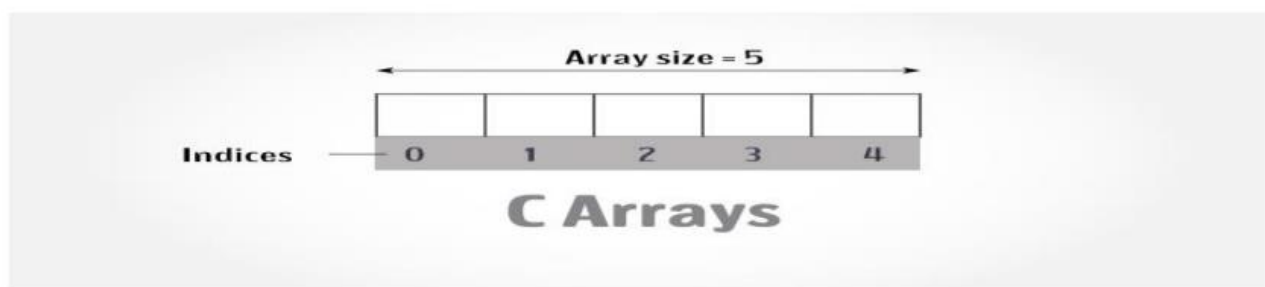
Data-type variable-name [size/length of array];

For example:

```
int arr[10];
```



```
int arr[ 5];
```



Here `int` is the data type, **arr** is the name of the array and 10 is the size of array. It means array **arr** can only contain 10 elements of `int` type. Index of an array starts from 0 to size-1 i.e first element of **arr** array will be stored at **arr[0]** address and last element will occupy **arr[9]**.

Initialization of an Array

After an array is declared it must be initialized. Otherwise, it will contain garbage value(any random value).
An array can be initialized at either compile time or at runtime.

Compile time Array initialization

Compile time initialization of array elements is same as ordinary variable initialization.

Syntax : data_type array_name[size]={v1,v2,...vn/list of values ;

Example

```
int age[5]={22,25,30,32,35};
```

	0	1	2	3	4
age	22	25	30	32	35

```
int marks[4]={ 67, 87, 56, 77 }; //integer array initialization
```

```
float area[5]={ 23.4, 6.8, 5.5 }; //float array initialization
```

```
int marks[4]={ 67, 87, 56, 77, 59 }; //Compile time error
```

Different ways of initializing arrays:

1: Initializing all specified memory locations

2: Partial array initialization.

3: Initialization without size

1 : Initializing all specified memory locations : If the number of values to be initialized is equal to size of array. Arrays can be initialized at the time of declaration. Array elements can be initialized with data items of type int, float, char, etc.

Ex: consider integer initialization

```
int a[5]={10,20,30,40,50};
```

During compilation, 5 contiguous memory locations are reserved by the compiler for the variable a and all these locations are initialized.

The array a is initialized as

a[0] a[1] a[2] a[3] a[4]

10	20	30	40	50
1000	1002	1004	1006	1008

If the size of integer is 2 bytes, 10 bytes will be allocated for the variable a.

2: Partial Array Initialization: partial array initialization is possible in C language. If the number of values to be initialized is less than the size of the array, then the elements are initialized in the order from 0th location. The remaining locations will be initialized to zero automatically.

Ex: Consider the partial initialization

```
int a[5]={10,15};
```

Even though compiler allocates 5 memory locations, using this declaration statement, the compiler initializes first two locations with 10 and 15, the next set of memory locations are automatically initialized to zero.

The array a is partial initialization as

a [0] a[1] a[2] a[3] a[4]

10	15	0	0	0
1000	1002	1004	1006	1008

How to access the elements of an array?

You can access elements of an array by indices/index. You can use array subscript (or index) to access any element stored in array. Subscript starts with 0, which means array name [0] would be used to access first element in an array.

In general array name [n-1] can be used to access nth element of an array. Where n is any integer number.

Example

```
float mark[5];
```

Suppose you declared an array mark as above. The first element is mark [0], second element is mark[1] and so on.

Few key notes:

- Arrays have 0 as the first index not 1. In this example, mark[0]
- If the size of an array is n, to access the last element, (n-1) index is used. In this example, mark[4]
- Suppose the starting address of mark [0] is 2120d. Then, the next address, a[1], will be 2124d, address of a[2] will be 2128d and so on. It's because the size of a float is 4 bytes.

Input data into array

As you can see, in above example that I have used „for loop“ and „scanf statement“ to enter data into array. You can use any loop for data input.

Code:

```
for (x=0; x<=19;x++)  
{  
    printf("enter the integer number %d\n", x);  
    scanf("%d", &num[x]);  
}
```

Reading out data from an array

For example you want to read and display array elements, you can do it just by using any loop. Suppose array is mydata [20].

```
for (int i=0; i<20; i++)  
{  
    Printf ("%d\n", mydata[x]);  
}
```

Exmaple

```
#include<stdio.h>  
  
#include<conio.h>  
  
void main()  
{
```

```
int i;

int arr[]={2,3,4}; //Compile time array initialization

for(i=0 ; i<3 ; i++) {

    printf("%d\t",arr[i]);

}

getch();

}
```

Output

2 3 4

Exmaple

1. include <stdio.h>
2. #include <conio.h>
3. void main(){
4. int i=0;
5. int marks[5]={20,30,40,50,60}; //declaration and initialization of array
6. clrscr();
- 7.
8. //traversal of array
9. for(i=0;i<5;i++){
10. printf("%d \n",marks[i]);
11. }
- 12.
13. getch();
14. }

Output

20

30

40

50

60

Runtime Array initialization

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large array, or to initialize array with user specified values.

Example

```
#include<stdio.h>

#include<conio.h>

void main()

{

int arr[4];

int i, j;

printf("Enter array element");

for(i=0;i<4;i++)

{

scanf("%d",&arr[i]); //Run time array initialization

}

for(j=0;j<4;j++)

{

printf("%d\n",arr[j]);

}

getch();

}
```

Two-Dimensional Arrays

The two dimensional array in C language is represented in the form of rows and columns, also known as matrix. It is also known as array of arrays or list of arrays.

The two dimensional, three dimensional or other dimensional arrays are also known as multidimensional arrays.

Declaration of two dimensional Array

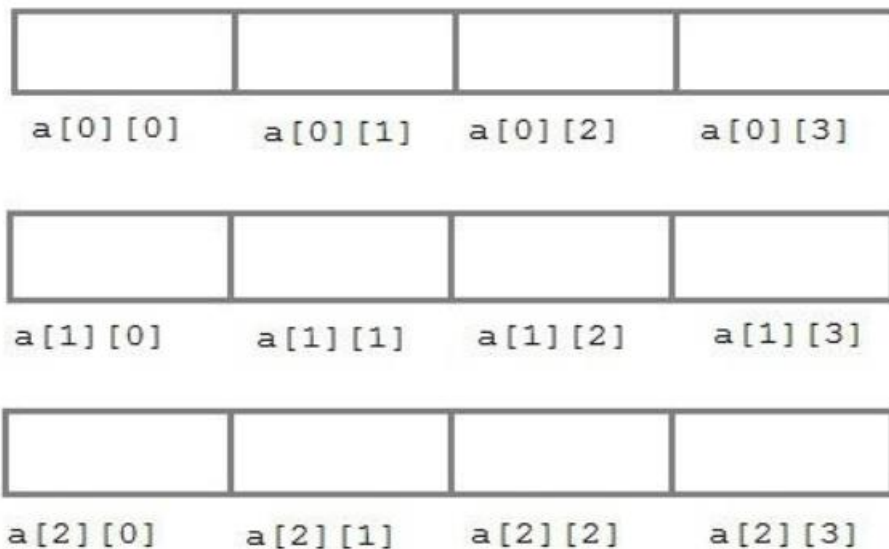
```
data_type array_name[size1][size2];
```

Example

```
int twodimen[4][3];
```

Example :

```
int a[3][4];
```



Initialization of 2D Array

```
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
```

Accessing Two-Dimensional Array Elements

An element in a two-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array.

Example

```
1. #include <stdio.h>
```



```
2. #include <conio.h>

3. void main(){

4. int i=0,j=0;

5. int arr[4][3]={ {1,2,3},{2,3,4},{3,4,5},{4,5,6} };

6. clrscr();

7. //traversing 2D array

8. for(i=0;i<4;i++){

9. for(j=0;j<3;j++){

10. printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);

11. }//end of j

12. }//end of i

13. getch();

14. }
```

Output:

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
```

```
arr[3][2] = 6
```

Example Write a C program Addition of Two Matrices

```
#include<stdio.h>

#include<conio.h>

void main()

{

int a[25][25],b[25][25],c[25][25],i,j,m,n;

clrscr();

printf("enter the rows and columns of two matrices:\n");

scanf("%d%d",&m,&n);

printf("\nenter the elements of A matrices");

for(i=0;i<m;i++)

{

for(j=0;j<n;j++)

scanf("\t%d",&a[i][j]);

}

printf("\nenter the elements of B matrices");

for(i=0;i<m;i++)

{

for(j=0;j<n;j++)

scanf("\t%d",&b[i][j]);

}

printf("\nThe elements of A matrices");

for(i=0;i<m;i++)

{
```

```
printf("\n");  
for(j=0;j<n;j++)  
printf("\t%d",a[i][j]);  
  
}  
  
printf("\nThe elements of B matrices");  
  
for(i=0;i<m;i++)  
  
{  
  
printf("\n");  
  
for(j=0;j<n;j++)  
  
printf("\t%d",a[i][j]);  
  
}  
  
printf("\nThe additon of two matrices");  
  
for(i=0;i<m;i++)  
  
{  
  
printf("\n");  
  
for(j=0;j<n;j++)  
  
{  
  
c[i][j]=a[i][j]+b[i][j];  
  
printf("\t%d",c[i][j]);  
  
}  
  
}  
  
getch();  
  
}
```

Write a C program Multiplication of Two Matrices.

```
#include<stdio.h>

#include<conio.h>

void main()

{

int a[25][25],b[25][25],c[25][25],i,j,m,n,k,r,s;

clrscr();

printf("enter the rows and columns of A matrices:\n");

scanf("%d%d",&m,&n);

printf("enter the rows and columns of B matrices:\n");

scanf("%d%d",&r,&s);

printf("\nenter the elements of A matrices");

for(i=0;i<m;i++)

{

for(j=0;j<n;j++)

scanf("\t%d",&a[i][j]);

}

printf("\nenter the elements of B matrices");

for(i=0;i<m;i++)

{

for(j=0;j<n;j++)

scanf("\t%d",&b[i][j]);

}

printf("\nThe elements of A matrices");

for(i=0;i<m;i++)
```

```

{
printf("\n");

for(j=0;j<n;j++)

printf("\t%d",a[i][j]);

}

printf("\nThe elements of B matrices");

for(i=0;i<m;i++)

{

printf("\n");

for(j=0;j<n;j++)

printf("\t%d",b[i][j]);

}

for(i=0;i<m;i++)

{

printf("\n");

for(j=0;j<n;j++)

{

c[i][j]=0;

for(k=0;k<m;k++)

c[i][j]=c[i][j]+a[i][k]*b[k][j];

}

}

printf("\nThe Multiplication of two matrices");

for(i=0;i<m;i++)

{

```

```

printf("\n");

for(j=0;j<n;j++)

printf("\t%d",c[i][j]);

}

getch();

}

```

Multidimensional Arrays

How to initialize a multidimensional array?

Initialization of a three dimensional array.

You can initialize a three dimensional array in a similar way like a two dimensional array. Here's an example

```

int test[2][3][4] = {

    { {3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2} },

    { {13, 4, 56, 3}, {5, 9, 3, 5}, {3, 1, 4, 9} }

};

```

Example

```

#include <stdio.h>

int main()

{

    // this array can store 12 elements

    int i, j, k, test[2][3][2];

    printf("Enter 12 values: \n");

    for(i = 0; i < 2; ++i) {

        for (j = 0; j < 3; ++j) {

            for(k = 0; k < 2; ++k ) {

                scanf("%d", &test[i][j][k]);

```

```

}

}

}

// Displaying values with proper index.

printf("\nDisplaying values:\n");

for(i = 0; i < 2; ++i) {

for (j = 0; j < 3; ++j) {

for(k = 0; k < 2; ++k ) {

printf("test[%d][%d][%d] = %d\n", i, j, k, test[i][j][k]);

}

}

}

return 0;

}

```

Output

Enter 12 values:

123456789101112

Displaying Values:

test[0][0][0] = 1

test[0][0][1] = 2

test[0][1][0] = 3

test[0][1][1] = 4

test[0][2][0] = 5

test[0][2][1] = 6

test[1][0][0] = 7

```
test[1][0][1] = 8
```

```
test[1][1][0] = 9
```

```
test[1][1][1] = 10
```

```
test[1][2][0] = 11
```

```
test[1][2][1] = 12
```

STRINGS:

String Concepts

String is an array of characters that is terminated by `\0` (null character). This null character indicates the end of the string. Strings are always enclosed by double quotes (" "). Whereas, character is enclosed by single quotes.

Or

In „C“ language the group of characters, digits, and symbols enclosed within double quotation (" ") marks are called as string otherwise a string is an array of characters and terminated by NULL character which is denoted by the escape sequence „\0“.

C Strings

Declaration of String: C does not support string as a data type. However, it allows us to represent strings as character arrays. In C, a string variable is any valid C variable name and it is always declared as an array of characters.

The general form of declaration of a string variable is :

Syntax: `char string_name[size];`

The size determines the number of characters in the string name.

Note: In declaration of string size must be required to mention otherwise it gives an error.

Ex: `char str[]; // Invalid`

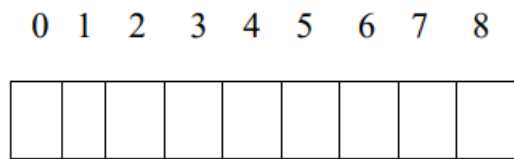
`char str[0]; // Invalid`

`char str[-1]; // Invalid`

`char str[10]; // Valid`

`char a[9]; //Valid`

Using this declaration the compiler allocates 9 memory locations for the variable a ranging from 0 to 8.



Here, the string variable a can hold maximum of 9 characters including NULL(\0) character.

Initializing Array string

Syntax : `char string_name[size]={“string” };`

Note: In Initialization of the string if the specific number of character is not initialized it then rest of all character will be initialized with NULL.

```
char str[5]={'5','+', 'A'};
```

```
str[0]; ---> 5
```

```
str[1]; ---> +
```

```
str[2]; ---> A
```

```
str[3]; ---> NULL
```

```
str[4]; ---> NULL
```

Note: In initialization of the string we can not initialized more than size of string elements.

Ex:

```
char str[2]={'5','+', 'A','B'}; // Invalid
```

Different ways of initialization can be done in various ways:

1 : Initilizing locations character by character.

2 : Partial array initialization.

3 : Intilization without size.

4 : Array initialization with a string .

1 : Initilizing locations character by character

Consider the following declaration with initialization,

Char b[9]={„C“,„O“,„M“,„P“,„U“,„T“,„E“,„R“};

The compiler allocates 9 memory locations ranging from 0 to 8 and these locations are initialized with the characters in the order specified. The remaining locations are automatically initialized to null characters.

C	O	M	P	U	T	E	R	\0
0	1	2	3	4	5	6	7	8

2 : Partial Array Initialization : If the characters to be initialized is less than the size of the array, then the characters are stored sequentially from left to right. The remaining locations will be initialized to NULL characters automatically.

Ex : Consider the partial initialization

int a[10]={„R“,„A“,„M“,„A“ };

The compiler allocates 10 bytes for the variable a ranging from 0 to 9 and initializes first four locations with the ASCII characters of „R“, „A“, „M“, „A“. The remaining locations are automatically filled with NULL characters (i.e,\0).

R	A	M	A	\0	\0	\0	\0	\0	\0
0	1	2	3	4	5	6	7	8	9

3 : Initialization without size : consider the declaration along with the initialization

char b[]={„C“,„O“,„M“,„P“,„U“,„T“,„E“,„R“};

In this declaration, The compiler will set the array size to the total number of initial values i.e 8. The character will be stored in these memory locations in the order specified.

b[0] b[1] b[2] b[3] b[4] b[5] b[6] b[7]

C	O	M	P	U	T	E	R
---	---	---	---	---	---	---	---

4) Array Initialization with a String : consider the declaration with string initialization.

char b[] = „COMPUTER“;

Here, the string length is 8 bytes. But, string size is 9 bytes. So the compiler reserves 8+1 memory locations and these locations are initialized with the characters in the order specified. The string is terminated by \0 by the compiler.

C	O	M	P	U	T	E	R	\0
0	1	2	3	4	5	6	7	8

The string “COMPUTER” contain 8 characters, because it is a string. It always ends with null character. So, the array is 9 bytes (i.e string length+1 byte for null character).

Reading and Writing Strings : The „%s“ control string can be used in scanf() statement to read a string from the terminal and the same may be used to write string to the terminal in printf() statement.

Example :

```
char name[10];  
  
scanf(“%s”,name);  
  
printf(“%s”,name);
```

Example:

1. #include <stdio.h>
2. void main ()
3. {
4. char ch[13]={‘c’, ‘p’, ‘r’, ‘o’, ‘g’, ‘r’, ‘a’, ‘m’, ‘m’, ‘i’, ‘\n’, ‘\0’};
5. char ch2[13] = “cprogramming”;
- 6.
7. printf(“Char Array Value is: %s\n”, ch);
8. printf(“String Literal Value is: %s\n”, ch2);
9. }

Output

Char Array Value is: cprogramming

String Literal Value is: cprogramming

Example:

```
#include <stdio.h>

int main()

{

    char name[20];

    printf("Enter name: ");

    scanf("%s", name);

    printf("Your name is %s.", name);

    return 0;

}
```

Output

Enter name: Dennis Ritchie

Your name is Dennis.

It is clear from the output that, the above code will not work for space separated strings. To make this code working for the space separated strings, the minor changed required in the scanf function, i.e., instead of writing `scanf("%s",s)`, we must write: `scanf("%[^\n]s",s)` which instructs the compiler to store the string `s` while the new line (`\n`) is encountered. Let's consider the following example to store the space-separated strings.

Example:

```
#include<stdio.h>

void main ()

{

    char s[20];

    printf("Enter the string?");

    scanf("%[^\n]s",s);

    printf("You entered %s",s);

}
```

}

Output

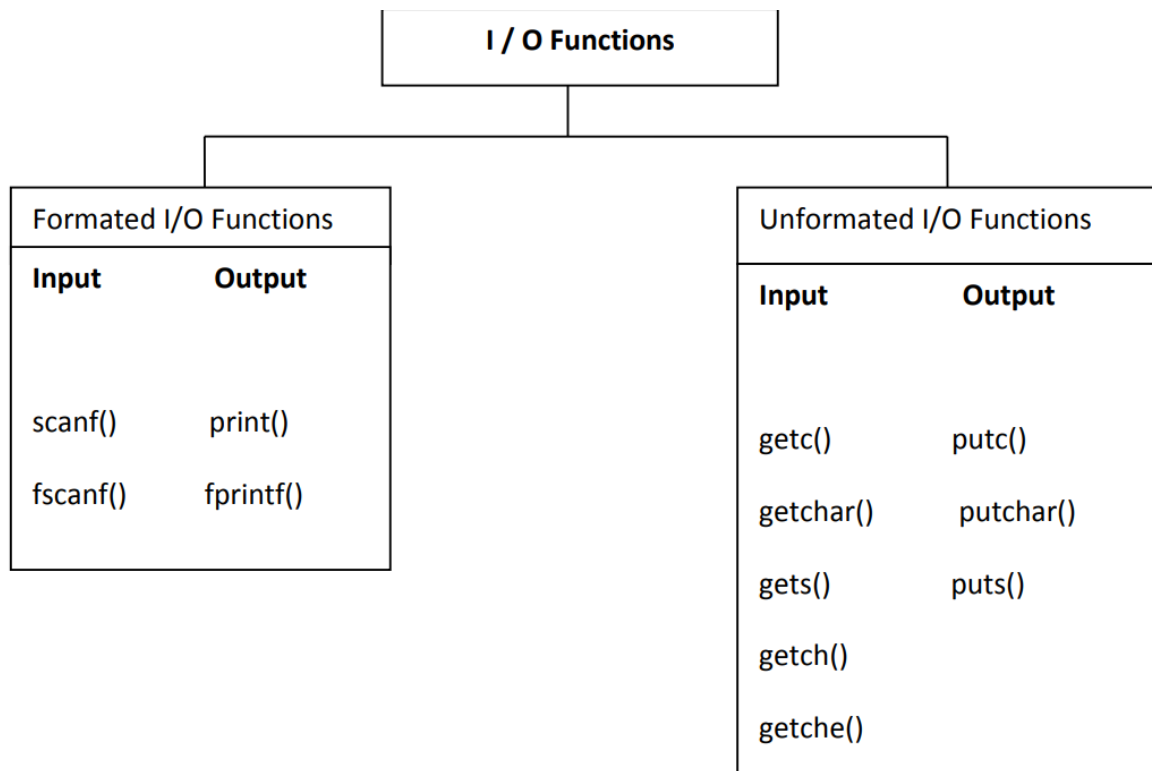
Enter the string? javatpoint is the best

You entered javatpoint is the best

String Input/output Functions

The strings can be read from the keyboard and can be displayed onto the monitor using various functions.

The various input and output functions that are associated with can be classified as



Unformatted I/O Functions

1 : getchar() function : A single character can be given to the computer using „C“ input library function getchar().

Syntax : char variable=getchar();

The getchar() function is written in standard I/O library. It reads a single character from a standard input device. This function does not require any arguments, through a pair of parentheses, must follow the statements getchar().

```
#include<stdio.h>
```

```

#include<conio.h>

#include<ctype.h>

void main()

{

char ch;

clrscr();

printf("Enter any character/digit:");

ch=getchar();

if(isalpha(ch)>0)

printf("it is a alphabet:%c\n",ch);

else if(isdigit(ch)>0)

printf("it is a digit:%c\n",ch);

else

printf("it is a alphanumeric:%c\n",ch);

getch();

}.

```

OUTPUT : Enter any character/Digit : abc

it is a alphabet:a

2 : putchar() function :The putchar() function is used to display one character at a time on the standard output device. This function does the reverse operation of the single character input function.

Syntax : putchar(character variable);

```

#include<stdio.h>

#include<conio.h>

#include<ctype.h>

void main()

{

```

```

char ch;

printf("Enter any alphabet either in lower or uppercase:");

ch=getchar();

if(islower(ch))

putchar(toupper(ch));

else

putchar(tolower(ch));

getch();

}

```

OUTPUT :Enter any alphabet either in lower or uppercase :a

A

3 : gets() : The gets() function is used to read the string (String is a group of characters) from the standard input device (keyboard).

Syntax : gets(char type of array variable);

Ex :#include<stdio.h>

#include<conio.h>

void main()

{

char str[40];

clrscr();

printf("Enter String name:");

gets(str);

printf("Print the string name%s:",str);

getch();

}

OUTPUT : Enter the string : reddy

Print the string :reddy

4 : puts() :The puts() function is used to display the string to the standard output device (Monitor).

Syntax : puts(char type of array variable);

Program using gets() function and puts() function.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
char str[40];
```

```
puts("Enter String name:");
```

```
gets(str);
```

```
puts("Print the string name:");
```

```
puts(str);
```

```
getch();
```

```
}
```

OUTPUT :Enter string name :

subbareddy

Print the string name

Subbareddy

String Manipulation Functions/ String Handling Functions

The various string handling functions that are supported in C language are as shown

String Function	Description
strlen(str)	Returns the length of the string str.
strcpy(str1,str2)	Copies the string str2 to string str1
strcat(str1,str2)	Append string str2 to string str1.
strlwr(str)	Converts the string str to lowercase
strupr(str)	Converts the string str to uppercase.
strrev(str)	Reverse the string str.
strcmp(str1,str2)	Compare two strings str1 and str2.

All these functions are defined in **string.h** header file.

1 : strlen(string) – String Length : This function is used to count and return the number of characters present in a string.

Syntax : var=strlen(string);

Ex : Progrm using strlen() function

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
char name[]="JBREC";
```

```
int len1,len2;
```

```
clrscr();
```

```
len1=strlen(name);

len2=strlen("JBRECECE");

printf("The string length of %s is: %d\n",name,len1);

printf("The string length of %s is: %d","JBRECECE",len2);

getch();

}
```

OUTPUT :

The string length of JBREC is : 5

The string length of JBRECECE is :8

Write a program to find the length of string with out strlen()

```
#include <stdio.h>

int main()

{

    char str[100];

    int i,length=0;

    printf("Enter a string: \n");

    scanf("%s",str);

    for(i=0; str[i]!='\0'; i++)

    {

        length++; //Counting the length.

    }

    printf("\nLength of input string: %d",length);

    return 0;

}
```

Output:

Enter a string:

PREPINSTA

Length of input string: 9

2 : strcpy(string1,string2) – String Copy : This function is used to copy the contents of one string to another string.

Syntax : strcpy(string1,string2);

Where

string1 : is the destination string.

string 2: is the source string.

i.e the contents of string2 is assigned to the contents of string1.

Copy String using strcpy()

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str1[20] = "C programming";
```

```
    char str2[20];
```

```
    // copying str1 to str2
```

```
    strcpy(str2, str1);
```

```
    puts(str2); // C programming
```

```
    return 0;
```

```
}
```

Copy String Without Using strcpy()

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```

// s1 is the source( input) string and
// s2 is the destination string
char s1[] = "GeeksforGeeks", s2[100], i;
printf("string s1 : %s\n", s1);
for (i = 0; s1[i] != '\0'; i++)
{
    s2[i] = s1[i];
}
s2[i] = '\0';
printf("String s2 : %s", s2);
return 0;
}

```

3 : strlwr(string) – String LowerCase : This function is used to converts upper case letters of the string in to lower case letters.

Syntax : strlwr(string);

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
char str[]="JBREC";
```

```
clrscr();
```

```
strlwr(str);
```

```
printf("The lowercase is :%s\n",str);
```

```
getch();
```

```
}
```

OUTPUT : The lowercase is : jbreC

Write a program to which converts given string in to lowercase without strlwr ()

```
#include<stdio.h>

#include<conio.h>

void main()

{

    char str[10];

    int index;

    printf("Enter the string:");

    scanf("%s",str);

    for(index=0;str[index]!='\0';index++)

    {

        if(str[index]>='A' && str[index]<='Z')

            str[index]=str[index]+32;

    }

    printf("After conversion is :%s",str);

    getch();

}
```

OUTPUT : Enter the string : SUBBAREDDY

After conversion string is :subbareddy

4 :strupr(string) – String UpperCase : This function is used to convert lower case letters of the string in to upper case letters.

Syntax :strupr(string);

Program usingstrupr() function.

```
#include<stdio.h>
```

```

#include<conio.h>

#include<string.h>

void main()

{

    char str[]="jbrec";

   strupr(str);

    printf("UpperCase is :%s\n",str);

    getch();

}

```

OUTPUT : UpperCase is : JBREC

Write a program to which converts given string in to uppercase withoutstrupr ()

```

#include<stdio.h>

#include<conio.h>

void main()

{

    char str[10];

    int index;

    printf("Enter the string:");

    scanf("%s",str);

    for(index=0;str[index]!='\0';index++)

    {

        if(str[index]>='a' && str[index]<='z')

            str[index]=str[index]-32;

    }

    printf("After conversionis :%s",str);

```

```
getch();
```

```
}
```

OUTPUT : Enter the string : subbareddy

After conversion string is : SUBBAREDDY

5. strcmp(string1,string2) – String Comparison : This function is used to compares two strings to find out whether they are same or different. If two strings are compared character by character until the end of one of the string is reached.

Case 1: when the strings are equal, it returns zero.

Case 2: when the strings are unequal, it returns the difference between ASCII values of the characters that differ.

a) When string1 is greater than string2, it returns positive value.

b) When string1 is lesser than string2, it returns negative value.

```
/* with using strcmp() */
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char str1[] = "abcd", str2[] = "abCd", str3[] = "abcd";
```

```
    int result;
```

```
    result = strcmp(str1, str2);
```

```
    printf("strcmp(str1, str2) = %d\n", result);
```

```
    result = strcmp(str1, str3);
```

```
    printf("strcmp(str1, str3) = %d\n", result);
```

```
    return 0;
```

```
}
```

Output

```
strcmp(str1, str2) = 1
```

```
strcmp(str1, str3) = 0
```

```
/* without using strcmp() */
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char Str1[100], Str2[100];
```

```
    int result, i;
```

```
qw    printf("\n Please Enter the First String : ");
```

```
    gets(Str1);
```

```
    printf("\n Please Enter the Second String : ");
```

```
    gets(Str2);
```

```
    for(i = 0; Str1[i] == Str2[i] && Str1[i] != '\0'; i++);
```

```
    if(Str1[i] < Str2[i])
```

```
    {
```

```
        printf("\n str1 is Less than str2");
```

```
    }
```

```
    else if(Str1[i] > Str2[i])
```

```
    {
```

```
        printf("\n str2 is Less than str1");
```



```

    }

    else

    {

        printf("\n str1 is Equal to str2");

    }

    return 0;

}

```

OUTPUT:

Please Enter the First String : cat

Please Enter the Second String : dog

str1 is Less than str2

6: strcat(string1,string2) – String Concatenation : This function is used to concatenate or combine, two strings together and forms a new concatenated string.

Syntax : strcat(sting1,string2);

Where

string1 : is the first string1.

string2 : is the second string2

when the above function is executed, string2 is combined with string1 and it removes the null character (\0) of string1 and places string2 from there.

Program using strcat() function.

```

#include<stdio.h>

#include<conio.h>

#include<string.h>

void main()

{

```

```
char str1[10]="jbrec";  
  
char str2[]="ece";  
  
strcat(str1,str2);  
  
printf("%s\n",str1);  
  
printf("%s\n",str2);  
  
getch();  
  
}
```

OUTPUT : jbrecece

Ece

Program using without strcat() function.

```
#include<stdio.h>  
  
void main(void)  
{  
  
    char str1[25],str2[25];  
  
    int i=0,j=0;  
  
    printf("\nEnter First String:");  
  
    gets(str1);  
  
    printf("\nEnter Second String:");  
  
    gets(str2);  
  
    while(str1[i]!='\0')  
  
        i++;  
  
    while(str2[j]!='\0')  
  
        {  
  
            str1[i]=str2[j];  
  
            j++;  
  
        }  
  
}
```

```

    i++;

}

str1[i]='\0';

printf("\nConcatenated String is %s",str1);

}

```

7. strrev(string) - String Reverse :This function is used to reverse a string. This function takes only one argument and return one argument.

Syntax : strrev(string);

Ex : Program using strrev() function

```

#include<stdio.h>

#include<conio.h>

#include<string.h>

void main()

{

char str[20];

printf("Enter the string:");

scanf("%s",str);

printf("The string reversed is:%s",strrev(str));

getch();

}

```

OUTPUT : Enter the string :subbareddy

The string reversed is : ydderabbus

Ex : Program using without strrev() function

```

int main()

{

char s[1000], r[1000];

```

```

int begin, end, count = 0;

printf("Input a string\n");

gets(s)

// Calculating string length
while (s[count] != '\0')

    count++;

end = count - 1;

for (begin = 0; begin < count; begin++) {

    r[begin] = s[end];

    end--;

}

r[begin] = '\0';

printf("%s\n", r);

return 0;

}

\

```

